UNIVERSITY
OF SUSSEX

Masters Thesis

On the Effectiveness of Deep Learning Techniques for Malware Detection and their Resiliance to Obfuscation

A thesis submitted in fulfilment of the requirements
for the degree of Master of Research in Advanced Artificial Intelligence

in the

University of Sussex
School of Engineering and Informatics

Author:
Henry Williams

Supervisor:
Jeff Mitchell

August 2025

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Malware is one of the most significant threats faced by users in the current digital landscape. The increasingly sophisticated nature of these programs, in terms of both capabilities and stealth, mean that this threat is only increasing as we become evermore reliant on digital infrastructure in our daily lives.

Therefore, detecting malware before it can cause damage is of high priority to security professionals. However, following the introduction of anti-virus software, malware developers of have employ various evasion techniques to avoid detection. This has led to a cat-and-mouse like game, where cybercriminals adopt new forms of evasion techniques, and security researchers developing new techniques to handle. This makes conventional approaches to malware detection challenging, due to its consistently evolving nature.

Machine learning however offers a potential means of dealing with this by finding ways to detect malware in spite of these techniques. In this investigation, we use various techniques across a number of domains, such as linear modelling, natural langauge processing, and image processing to investigate both their performance at detecting malware targeting the windows operating system, and their resiliance to obfuscation.

The research questions we seek to answer in this work are as follows:

RQ1: *Are modern natural language processing techniques, such as language models and pretraining effective at identifying malware?*

RQ2: *Does pretraining improve the accuracy and reliability of malware detection models based on techniques from natural language processing?*

RQ3: *To what extent are static malware detection models resiliant to obfuscation?*

1.1 Professional Considerations

This research is conducted in strict accordance with the BCS Code of Conduct (British Computing Society, 2022), ensuring adherence to the ethical, legal, and professional standards expected of computing professionals. The work is structured to protect the public interest, maintain professional competence and integrity, uphold obligations to the relevant authority, and safeguard the reputation of the profession.

1.1.1 PUBLIC INTEREST

We conduct our research with regard to the safety, rights, and welfare of the public to ensure our work is in their interest. Samples of malicious programs will be handled in a secure environment,

with safeguards to protect against execution on both our own hardware, and that of others to avoid accidental infection. Furthermore, proprietary programs subject to copyright law will not be distributed to ensure the intellectual property rights of third parties are respected. Our research will be conducted without discrimination on the basis of ethnicity, sex, religion, age or other protected characteristsics, and we distribute all materials used to conduct this investigation to support transparency, reproducibility, and to benefit the wider research community.

1.1.2 PROFESSIONAL COMPETENCE AND INTEGRITY

We conduct our research within the verified level of competence of the authors, and avoid misrepresenting our skills and expertise. The project itself contributes to the continued professional development of our skills in Artificial Intelligence, and Cybersecurity. We employ established best practices in both fields where appropriate, and comply with all relevant UK legislation, including data protection, intellectual property, and cyber-security. Diverse perspectives are actively sought, and the authors welcome constructive criticism to ensure the quality and robustness of the work. No element of this research will cause harm to the property, reputation, or the livelihood of others through false or negligent actions.

1.1.3 DUTY TO THE RELEVANT AUTHORITY

To ensure the duty of the authors to relevant authorties, we conduct this research in compliance with the policies and procedures of the University of Sussex, and any other governing bodies. Any potential conflicts of interest are avoided, and disclosed in cases where they are unavoidable. We accept any professional responsibilities for our work, and that of others involved with this project. Results and findings will be reported accurately, and truthfully, without distortion, omission, or misrepresentation.

1.1.4 DUTY TO THE PROFESSION

The authors acknowledge their responbility to uphold the credibility and reputation of the computing profession, and we pledge to take no actions that will take it into disrepute.

Chapter 2

Background

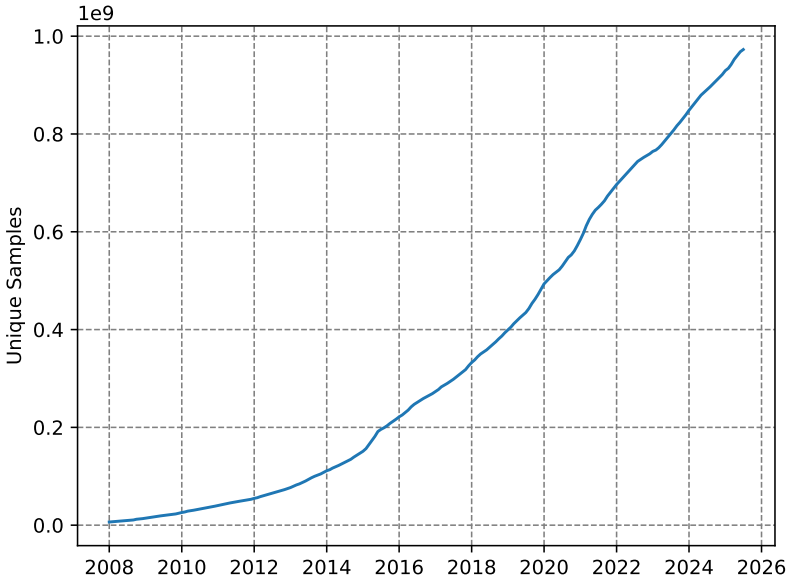


Figure 2.1: Observed growth in the number of unique forms of malware over time. Data taken from AV-TEST - The Independent IT-Security Institute, 2025

Malicious software, also known as malware, is defined as software that have an adverse impact on the typical operation of a system. This could include the extraction of sensitive information, hijacking of resources, and modification of information. They are considered one of the most pervasive threats faced by both individuals and organisations in the current digital threat landscape (European Union Agency for Cybersecurity, 2024; Alenezi et al., 2020).

This can been seen in infamous cyberattacks such as the 2022 Viasat hack (Quiquet, 2023), and the 2017 WannaCry incident (National Audit Office (NAO), 2017; Akbanov et al., 2019). In both of these cases, sophisticated malware were used to disrupt critical infrastructure, including communication systems across Europe, and services used by the National Health Service.

In recent years, the sophistication (Alenezi et al., 2020; Baezner et al., 2017) and prevalance (Figure 2.1) of malware have increased. Therefore, there has been a great deal of interest into advanced techniques for the reliable detection of malware (M. et al., 2023; Maniriho et al., 2024; Aboaoja et al., 2022) within the information security field. In this chapter, we provide a background on characteristics commonly associated with malware, and discuss various techniques proposed by security researchers to better handle this threat.

2.1 Malware

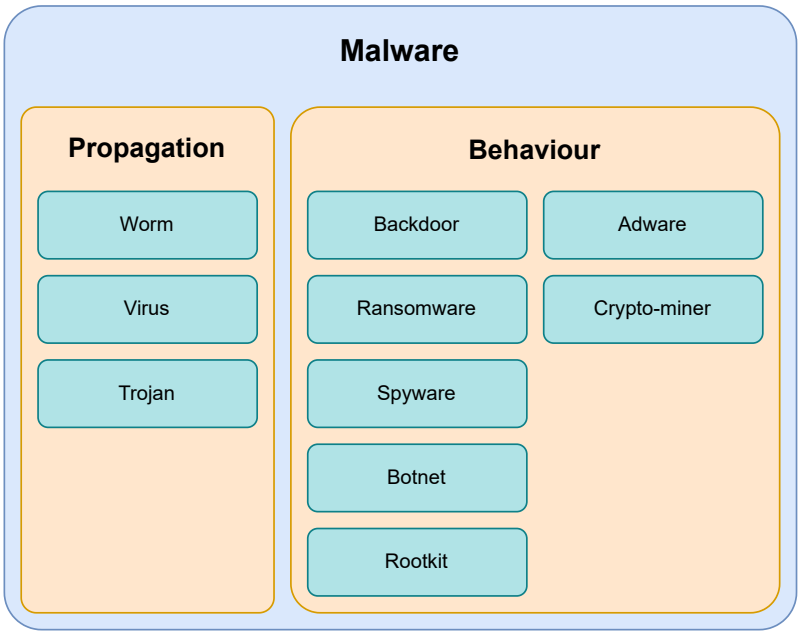


Figure 2.2: Categories of Malware, including both the means of propagation and behaviour on an infected host.

As outlined in the previous section, malware is a form of software that has a detrimental impact on the expected operation of an infected system. However both this and more formal definitions, such as the one presented in (Kramer et al., 2010), fail to define what is meant by harmful. Therefore, we define "harmful" behaviour as any actions in violation of the Confidentiality-Integrity-Availability (CIA) triad (Cochran, 2024). This is a principle that aims to guide the development of security policies to ensure the security of digital systems, and the information stored on them. It states that a system must be inaccessible to unauthorised users (Confidential), accessible to authorised users (Available), and the protected against the modification or deletion of the information it stores (Integrity). Malware provides a comprehensive means of violating this triad of security principles, and it is for this reason we use it in our definition.

Malware is categorised based on its behaviour, and how it spreads between hosts. We present the following taxonomy based on the work of Maniriho et al., 2024 shown in Figure 2.2 that outline the different kinds of malicious behaviors, and propagation techniques. Note that categories within this taxonomy are not mutually exclusive, as malware often belong to multiple categories at once.

- **Worm** — Malicious software that automatically propagates throughout a network.
- **Virus** — Programs that modify other programs to replicate themselves throughout an infected host.
- **Trojans** — Malware that disguises itself as a legitimate program to infect a host.
- **Backdoor** — Malware that provides covert remote access to infected systems.
- **Ransomware** — Malware that denies access to systems or information until a fee is paid.
- **Spyware** — Malware that spy on the user without their knowledge, often extracting information such as credentials, keyboard logs, and live screenshots of the user's activity.

- **Botnet** — Malware that integrate infected systems into a coordinated network of agents.
- **Rootkit** — Malware providing privileged access to an infected system.
- **Adware** — Malware that display advertisements on the host machine.
- **Crypto-miner** — Malware that hijack the user's hardware to mine cryptocurrencies.

2.1.1 EVASION TECHNIQUES

The earliest forms of malware detection used unique signatures associated with malicious programs(A. Saeed et al., 2013). To counter this, the developers of malicious software began using evasion techniques that aim to decieve these systems to maximise the amount of potential damage their software is able to do before it is identified and removed. These include obfuscation, and environmentally aware execution. In the following sections, we breifly discuss these techniques, and how they are effective in bypassing detection.

Obfuscation

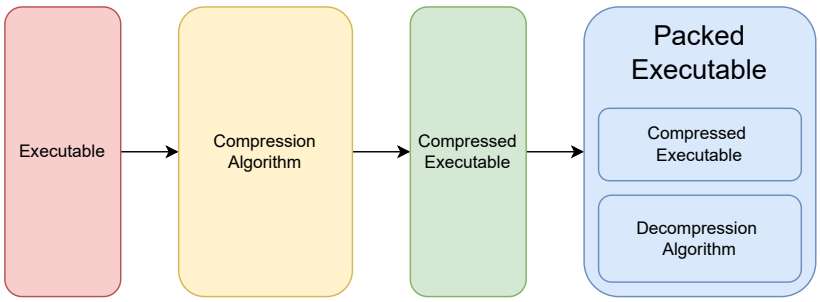


Figure 2.3: How binary packing can be used to reduce the size of an executable program, and obscure its behaviour.

Obfuscation refers to the act of making something more challenging to understand to an external observer. In the context of malware, it includes techniques which alter the syntactic structure of a program, yet maintain the original behavioural characteristics.

Register re-assignment is one of the earliest forms of obfuscation in which the registers used by the program are randomly reallocated (Balakrishnan et al., 2005). Registers are small areas of memory within the CPU that store information used during the program's execution. By changing which registers are used to store this information, the physical structure of the program is changed without impacting its behavior. This defeats simple malware detection solutions that use the unique cryptographic hashes of a program as its signatures, as a unique representation of a program will give a distinct hashes that has not been seen before.

We can also modify the structure of a program by inserting instructions, or functions which either have no impact on the its state, or are never executed. This is known as dead-code insertion, and is a relatively common form of obfuscation, as it can provide vastly different representations of the same program. However, so-called "dead-code" can easily be identified and removed, by either human experts, or a call flow graph.

Code transposition modifies the order of code the program (Christodorescu et al., 2004) to obscure its signatures by moving small units of code known as basic blocks. To ensure the program behaves as expected, all references to the relocated sections are updated to its new location. This again evades detection by these early signature based methods, as many unique representations of the same program can be generated.

Despite the ability of these techniques to trick early anti-malware approaches, more sophisticated approaches are more resiliant to these forms of obfuscation. These approaches use an intermediate representation of the program, that normalise the structure of a program resulting in a representation that is indentical to the un-obfuscated representation. To decieve these advanced approches, more sophisticated forms of obfuscation are needed. One such example of this is known as binary packing, where the malicious portion of code is encoded and stored as data within another program that decoded and executes the payload (Figure 2.3). Encoding can take many forms, such as encryption or compression, but they typically make it challenging to retrieve the decoded payload without running the program. However, the portion of the packed binary responsible for decoding and execution will often remain constant, meaning it can be used for identification. Some malware developers employ the aforementioned obfuscation techniques to obscure the decoder and avoid detection which are known as Oligomorphic and Polymorphic malware (You et al., 2010).

Environmental Awareness

The techniques demonstrate the challenges of relying on the static strucuture of a program to determine its intent, and it is for this reason that dynamic features are considered to be more robust indicators of malicious programs. This is because behaviour of a program is invariant to these forms of obfuscation. Dynamic features are observed at execution time and include sequences of system API calls, modifications to the filesystem and system configuration, and internet traffic. However, this requires running a potentially malicious program, which could have harmful effects on the host system. Therefore, the execution of a potentially malicious program will occur within a sandbox, an environment where harmful actions can occur with no implications on the security of the system. This sandbox is typically a virtual machine that is representative of the software's intended target.

To decieve these approaches, the malware developers will have their code check if it is running within a virtual machine before it executes. Common inidicators of a virutal machine used by malicious programs include system hardware characteristics (CPU core count, RAM size, etc), the presence of shared libraries used to communicate between the sandbox and its host, and the number of processes running on the system. If the program determines that it is likely to be running within a sandbox, it will refuse to execute the malicious portion of the code, and will exit. This prevents the observer from gaining any information about its behavioural characteristics, preventing the creation of dynamic malicious signatures.

2.2 Related Work

Malware detection is a challenging problem, and many approaches have been proposed since the first forms of malicious software were seen in the 1980s. The earliest form of malware detection used a database containing the unique signatures of malicious programs. While similar signature based approaches are still in use today, there is much interest in advanced methods, that are robust to evasion techniques, and that able to identify novel forms of malware (M. et al., 2023; Merabet et al., 2019; Aboaoja et al., 2022; Idika, Mathur, 2007). These advanced techniques employ a wide variety of technologies, such as multiple sequence alignment of API call sequences (Ki et al., 2015), machine learning (Kamboj et al., 2023), and deep learning.

These techniques typically fall into one of two categories, based on how the features used to classify programs are extracted which are as follows.

- **Static** — Features extracted from the structure of a program
- **Dynamic** — Features extracted during the execution of the program

Static approaches use features extracted from the contents of an executable as opposed to its behavior during execution. Common features used to identify malware include greyscale images of

the executable (Ni et al., 2018; Jain et al., 2020; Habibi et al., 2023), printable strings (Tian et al., 2009; Singh et al., 2020; Schultz et al., 2001), and sequences of instructions (Kakisim et al., 2022; Wang et al., 2021; Darem et al., 2021).

In Schultz et al., 2001, the authors use static features such as printable ASCII strings to train various machine learning classifiers to detect malware. In their experiments, they found that the Naive Bayes algorithm was best suited to the problem, achieving 97% classification accuracy. This is one of the first papers that introduce the usage of machine learning in this domain, and the high accuracy of their experiment shows its potential over conventional signature or heuristic methods. However, the dataset used in this work was small, and featured an unbalanced class distribution of 3265 malicious programs, and only 1001 benign which would likely have an adverse effect on the generalisability of their proposed models. Furthermore, malware often obfuscate strings that are decoded during execution, which would be sufficient to defeat their proposed approach.

Much of the literature relating to static malware detection use convolutional neural networks to distinguish between malicious and benign programs (Ni et al., 2018; Jain et al., 2020; Habibi et al., 2023). In Aslan et al., 2021, the authors represent an executable as a greyscale image, and train a convolutional neural network (CNN) to detect malware. While their proposed approach is able to effectively distinguish between executable classes, CNNs require constant input size. Therefore, resizing of the image representation is necessary. Malicious programs are often smaller than benign programs, meaning that in datasets where there is significant variation in size between classes, compression artifacts may be more prevalant in once class, leading to the model developing an insufficient solution. In Sharma et al., 2019, the authors also use a CNN to detect malware. Instead of using a 2-dimensional CNN however, the authors opt to use a 1-dimensional variant. This led to better performance than other CNN based approaches, as the vertical spatial dependence, which is largely irrelevant in malware detection, is removed. However, it still has the same requirement as the previously discussed approach.

Instruction sequences have also been explored as static features in malware detection. For example, in (Attaluri et al., 2009; Runwal et al., 2012), the authors utilise hidden markov models and achieve promising results. As these models consider what actions are being performed by the host system without having to execute the program, they offer a solution that we believe to be more resilient to obfuscation than other static approaches.

Extending upon the idea of utilising instruction sequences for malware detection, (Demırcı et al., 2022) outline an approach in which they propose the use of stacked bidirectional Long-Short Term Memory (BiLSTM) and pre-trained language models including BERT (Devlin et al., 2019), and GPT-2 (Radford et al., n.d.). In their investigation, the BiLSTM model was able to achieve 98% classification accuracy whereas the pre-trained transformer models achieved only 77%. Transformer models have been shown to be a powerful technique for sequence classification problems, and as such, this result is surprising. We believe this to be a result of the corpora used to pre-train the transformer models used in their investigation, which are comprised of mostly english texts. The understanding of these models would thus be limited to languages similar to english, and may struggle to understand the intricacies of assembly language, impacting their ability to reliably classify malware.

Previous research has also established the use of pre-trained language models in malware detection on the metadata of an executable. In Rahali et al., 2021, the authors fine-tune BERT on text gathered from the manifest file found in android applications. This file contains information pertaining to the activities, background tasks, event listeners, and permissions granted to an application. Their model was able to achieve 97% binary classification accuracy, in comparison to (Demırcı et al., 2022) which was only able to achieve 77% with the same model, trained on instruction sequences. We believe this discrepancy to be a result of the language found in these manifest files being more closely related to the pre-training corpora of the BERT model. This would enable their model to better understand the information found in the manifest, and its use in distinguishing between benign and malicious executables. Their approach however, is limited to environments where a file con-

taining metadata are required, such as mobile operating systems, making it unsuitable for desktop malware detection.

The performance of static malware detection approaches vary, depending on the detection method used, the availability of data, and other factors, but it is well established that they are particularly vulnerable to obfuscation. In Bacci et al., 2018, the authors investigate the impact of obfuscation on the performance of static and dynamic malware detectors on android devices. They found that while dynamic methods are not significantly impacted by the presence of obfuscation, static techniques are more affected. However, by training static methods on obfuscated programs, it is possible to mitigate this problem to some extent.

Furthermore, dynamic methods generally tend to be more capable than static methods (Damodaran et al., 2017). This is somewhat to be expected, as behaviour is more indicative of malicious actions, especially when obfuscation is used to hide a program's intent. However, dynamic methods require a sandbox to execute a program for analysis, without causing damage to a host. This typically takes the form of a remote, virtual machine as running such a sandbox locally would have a significant computational cost. As a result of this, dynamic methods require both significant compute resources, and an internet connection, which is not guaranteed. It is for this reason that static methods, that can identify potentially malicious programs, without an internet connection is needed for enhanced protection from this threat.

In addition to the high resource requirements associated with dynamic feature extraction, sophisticated evasion techniques such as environmental awareness and living off the land provide malware developers with the capabilities to obscure the dynamic features. For example, by either preventing execution within virtual machines, or using trusted binaries already located on a host system. Furthermore, novel obfuscation techniques, such as the one proposed by Li et al., 2023, is able to obscure which system API calls are made by a program. This is a widely used dynamic feature for detecting malware, and the proposed technique enables the malware to call these system APIs without triggering system hooks used to track which methods are called during a program's execution. Therefore, despite offering better performance than static methods, dynamic methods are also vulnerable to evasion techniques.

2.3 Datasets

Research into malware detection depends on the availability of large scale, high quality datasets containing both malicious and benign software. However, very few of these datasets exist, and those that do often provide features derived from the programs as opposed to the executables themselves. For example, the EMBER dataset (Anderson et al., 2018) contains over a million samples of static features derived from each executable, however the binaries themselves are not distributed. Other datasets, such as BODMAS (Yang et al., 2021), despite offering fewer samples than EMBER, provide access to the executables. However, due to copyright limitations, only malicious binaries are provided.

When curating these datasets, malicious samples are easy to download en-masse through websites such as VirusShare[1] and VirusExchange[2]. These websites store large collections of malware seen in-the-wild by security professionals. However, similar large scale collections of benign software are less available, likely due to copyright protections. We have seen a number of approaches for curation of benign software in the literature. For example, by downloading android applications from the Google play store (Karbab et al., 2018), scraping free software websites (Li et al., 2022), and extracting system executables. These approaches however are somewhat limited by the number of programs available on both free software websites, and the system itself. Furthermore, these executables are not guaranteed to be free of malware. Another promising approach for collecting

[1] `https://virusshare.com`

[2] `https://virus.exchange/`

benign windows software is the downloading of repositories used by windows package managers such as chocolatey and Winget as they provide many packages that have been verified as being free of malware. However, they often distribute installers rather than the executables themselves, meaning each package has to be installed before it can be used in the dataset.

2.4 Future Research Directions

Our analysis of the literature relating to this field, has found that there are a number of promising areas of investigation. Static approaches, while less robust to structural obfuscation, are less resource intensive and are able to run on a users machine without an internet connection. Dynamic features offer better performance than its static counterparts, yet are still vulnerable to advanced evasion techniques that may render them less effective than static. Furthermore, the requirements associated with dynamic feature extraction are a significant barrier to their deployment, and demonstrate the need for efficient and effective static methods for detecting malware.

Furthermore, we identify a gap in the literature relating to the use of modern natural language processing techniques such as pretrained transformer models in malware detection. While these models have been used in this context, they use the natural langauge features of these programs, such as the ascii strings, and program metadata. We will investigate the use of these technqiues when trained to develop an understanding of the program itself to see if these modern NLP techniques are suited to reasoning about executables for their use in detecting malware.

In addition to this, we will investigate if static methods learn to detect malware based on the presence of obfuscation, as it seems that the prevalance of obfuscated malware is significant in real world cybersecurity incidents. This will develop a better understanding of the issues associated with static malware detection.

Chapter 3

Methodology

In this investigation, we evaluate the effectiveness of a custom RoBERTa model in static malware detection. The model is trained to distinguish between malicious and benign opcode sequences using modern natural langauge processing techniques such as pretraining, which are then used to identify malicious programs. We compare this model against both linear models, such as decision trees, and other static malware detection methods seen in the literature, such as Convolutional Neural Networks. Finally, we evaluate the resiliance of the opcode sequence classificatio, and the CNN image classification models against obfuscated malware to assess their inner workings.

3.1 Dataset

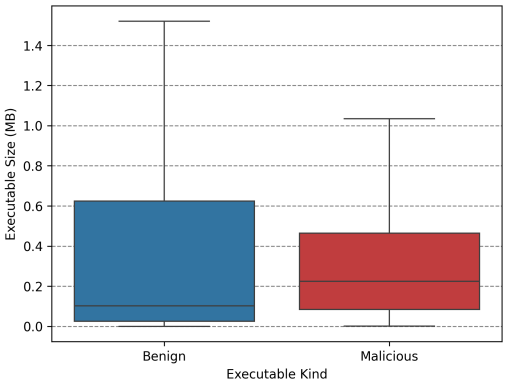
Our dataset consists of 11115 windows PE32 files (See appendix A.6), with a class distribution shown in fig. 3.1c. Malicious files are gathered from VirusShare[1], which contains samples from real-world cybersecurity incidents, providing approximately 90 million samples of which nearly 6000 were randomly selected. The benign executables were collected from the personal devices of the authors, and were scanned using Microsoft defender to ensure no mislabelled samples were present. This includes various software packages, games, developer tools, system programs, and other miscellaneous executables to ensure the dataset is representative of the kinds of benign programs commonly seen on a potential end users' machine.

We opt to investigate windows malware detection, due to its widespread usage, and therefore high availability of malware that target it. Malware that target other operating systems do exist, but there are significantly fewer samples and therefore were not selected as the target for this investigation.

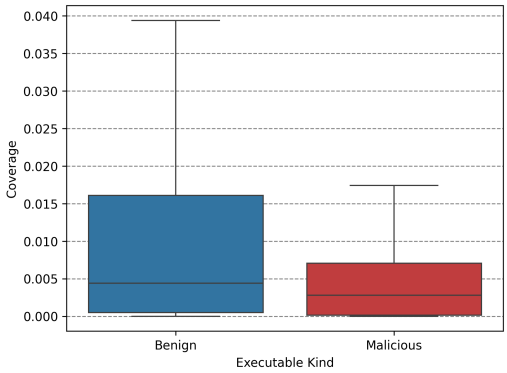
The dataset demonstrates wide variation in characteristics across classes. For example, we find that malicious programs are often much smaller than benign executables as shown in fig. 3.1a. We believe this to be because benign programs typically provide more functionality, such as graphical user interfaces, telemetry, and compatability, which increase the size of the program. Malicious software on the other hand, are developed with a single goal in mind, reducing its size.

We hypothesize that both our dataset, and other malware detection datasets containing malicious software will demonstrate variation in the prevalance of obfuscation across classes. To investigate this, we perform static analysis of all samples in our dataset to find the code coverage using Rizin (Rizin Organization, 2025) to find the analysis coverage. This metric gives the ratio of code analysed by the tool, to the total amount of code in an executable. In obfuscated programs, this result is typically much lower than non-obfuscated executables. The results of this analysis is presented in Figure 3.1b, which supports our hypothesis, and indicates malicious samples are more frequently obfuscated than benign samples.

[1] `https://virusshare.com/about`



(a) Distribution of executable size across benign and malicious programs



(b) Distribution of code analysis coverage of benign and malicious programs

Class	Count
Malicious	5792
Benign	5323
Total	11115

(c) Distribution of classes with our dataset.

Figure 3.1: Description of our dataset, including the size of programs (a), the extent of obfuscation (b), and the total number of samples in each class (c).

The dataset is then split to produce a training set, and evaluation set at a ratio of 80% to 20% respectively. To ensure consistent class distribution between these sets, samples are shuffled prior to splitting.

3.1.1 OBFUSCATION EXPERIMENT DATASET

Class	Count
Malicious	60
Benign	61
Total	121

Figure 3.2: Distribution of classes with our obfuscation experiment dataset.

To investigate the impact of obfuscation on malware detection, we also curate a secondary dataset, consisting of unobfuscated executables. We use the work presented in (Rokon et al., 2020) to identify github repositories containing the source code of malware. The authors provide a list of URLs to these repositories from which select windows programs, that provide an executable, with no mention of obfuscation in its description. Benign samples consist of Windows system binaries not present in our original dataset. We identify a collection of approximately 60 windows XP system binaries which are unlikely to include obfuscation for this secondary dataset. This results in a selection of 121 programs, each of which are free of obfuscation for use in our later experiment into the resiliance of machine learning algorithms to obfuscation.

3.2 Metrics

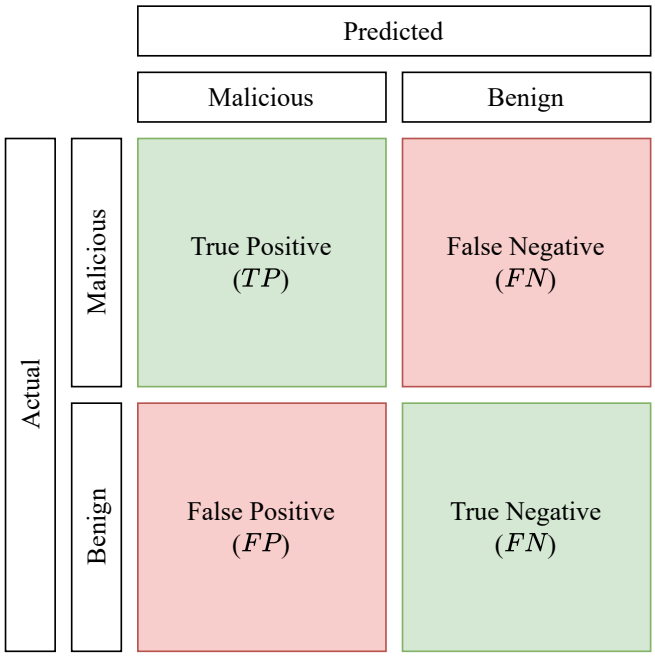


Figure 3.3: Confusion Matrix outlining the kinds of classification results in malware detection.

In our investigation, malware detection is presented as a binary classification problem, where our models aim to distinguish between malicious and benign software. We quantify the performance of the various approaches tested in this investigation, we find the accuracy, f1-score, precision, and recall (See eq. (3.1)). These metrics are based on the different kinds of results a binary classifier can give as shown in fig. 3.3.

Accuracy is the ratio of correct classifications to the number of samples tested, where 1 would be a perfect classifier for a given set of samples. However, the value of this metric can be misleading in unbalanced datasets.

Precision and recall both operate on the class-level, therefore to reduce these metrics to a single number, we typically take the average of these values for each class in the dataset, known as macro-averaging. Precision is definied as the ratio between the number of correctly identified samples, to the total number of samples predicted as belonging to a given class. Recall on the other hand is the number of correct predictions, to the number of samples in given class. F1 score, acts as an aggregation of these values, and is given by the harmonic mean.

$$\begin{aligned} \text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\ \text{Precision} &= \frac{TP}{TP + FP} \\ \text{Recall} &= \frac{TP}{TP + FN} \\ \text{F1-Score} &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \end{aligned} \tag{3.1}$$

3.3 Baseline

Similarity	Malicious	Benign
Malicious	0.1428	0.0931
Benign	0.0931	0.1217

(a) Cross-class external function similarity

Similarity	Malicious	Benign
Malicious	0.2249	0.1606
Benign	0.1606	0.1643

(b) Cross-class DLL similarity

Figure 3.4: Similarity between classes based on external functions and DLLs

We firstly establish a baseline using conventional machine learning algorithms on the lookup table associated with each program. This lookup table tells the program where to find external references, such as libraries and functions exist in memory. These external references often functionality that would otherwise be inaccessible to the program. For example, to interact with the hosts file system, the lookup table will include a reference to `KERNEL32.dll`, which provides the functionality requested by the program. We can extract all functions and libraries used by an executable with static analysis tools such as radare2(Radare Organization, 2025). References to functions and libraries that occur once across the dataset are removed as approximately half of all DLLs and functions are seen exactly once across the entire dataset. This gives two binary feature vectors for each sample in the dataset, telling the classifiers which external functions, and libraries are used by the program. We test 3 machine learning algorithms to establish this baseline, which are support vector classifiers, logistic regression, and decision trees.

We also use the feature vectors to quantify the similarity of programs in our dataset with the pairwise cosine similarity function (Equation 3.2). We then find the average similarity between classes which are presented in Figure 3.4. From this, we can assume that functions and libraries used by the malicious programs are more similar than the ones used by benign programs. Furthermore, functions used by malicious programs are vastly different to functions used by benign programs.

$$S(\mathbf{A}, \mathbf{B}) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} \tag{3.2}$$

3.4 RoBERTa for Opcode Sequence Classification

```
ret lea mov add add lea mov add add nop
```

Figure 3.5: An example sequence of opcodes

In recent years, transformer-based architectures for natural language processing have been shown to be highly effective across a wide variety of language based tasks. A significant advancement within the field was the development of the Bidirectional Encoded Representation from Transformers model, known as BERT (Devlin et al., 2019). This model uses an encoder-only architecture, based on the multi-head self-attention mechanism (Vaswani et al., 2023) in conjunction with a pretraining stage to develop a comprehensive understanding of language. Pretraining consists of two tasks, masked language modelling where the model predicts hidden tokens based on their surrounding context, and next sentence prediction. The pretrained model can then be fine-tuned on a downstream problem such as sequence classification, question answering, and general language understanding. At the time of publication, it achieved state-of-the-art performance in the General Language Understanding Evaluation (GLUE), Stanford Question Answering Dataset (SQuAD), and Situations With Adversarial Generations (SWAG) benchmarks and has been used extensively throughout the field.

Fine-tuning of the pretrained model allows researchers to solve tasks with fewer training samples, short training periods, and less intensive compute requirements. RoBERTa extends upon the original BERT model by expanding the number of parameters, removing the next sentence prediction task from the pretraining stage, optimising its hyperparameters, and training on a larger corpus.

We develop a custom RoBERTa model, trained on opcode sequences for malware detection. Opcodes refer to the operation to be performed by the CPU, such as `add`, `cmp`, `mov`, written in a human readable form. To generate these sequences, we firstly use radare2 to extract the disassembly of a program (See algorithm 1), and remove the operands of each instruction (algorithm 2) to reduce the vocabulary size and therefore the computational resources needed to train the model. This gives a sequence of opcodes representing each program within our dataset, as shown in fig. 3.5. The sequences are then tokenized at the word level where each opcode is assigned a single token.

We train four such models that take a tokenized opcode sequence of some length L. The first model, is pretrained on masked langauge modelling. We use this pretrained model to train a sequence classification model, refered to as the "pretrained" model. We also train another pretrained sequence classification model which has had the weights in its attention layers frozen, refered to as the "frozen" model to determine if the representations produced by the attention layers are sufficient in distinguishing between malicious and benign opcode sequences. Finally, we also train a RoBERTa sequence classification model from scratch, to assess if the pretraining step is necessary.

Our models share the architecture shown in Table 3.1 that was found to be the optimal configuration after an extensive hyperparameter sweep of 90 runs. Training was performed with a single Nvidia A40 GPU, and 16GB of ram with a batch size of 128 for a single epoch, using the AdamW optimizer, and Cross-Entropy loss function provided by Pytorch.

For each opcode sequence, the model predicts two values, the likelihood of the sequence being malicious, or benign where the larger of these two values is its predicted class. However, the full opcode sequence of a program is too long to be used as the models input. Therefore, we split the full sequence of opcodes into multiple sub-sequences of some fixed length, and aggregate the logits to classify a program. We test both raw logit mean aggregation, softmax logit mean aggregation, and majority voting, and found raw logit mean aggregation to be most effective.

Parameter	**Value**
Sequence Length	32
Hidden Layer Size	512
Hidden Layers	7
Attention Heads	4
Intermediate Layer Size	2048
Activation Function	GELU
Hidden Dropout Rate	0.01
Attention Dropout Rate	0.001

Table 3.1: RoBERTa model parameters

3.5 CNN for Image Classification

In addition to the RoBERTa sequence classification model, we also evaluate an image based approach. We use the EfficientNet-Bo Convolutional Neural Network architecture (Tan et al., 2020), due to its computational efficiency and high performance on image classification problems. The model is trained from a random initialisation, and the final layer of the model is modified to output a binary

class likelihood values. Programs are converted to greyscale images using the procedure outlined in section A.3, converted to RGB, and resized to 320×320 pixels before it is used by the model.

As previously established, the size of benign programs is typically much higher than malicious programs. Therefore, benign images are resized to a greater extent to that of malicious images. Resizing an image will introduce artifacts to the image, such as anti-aliasing or blurring. Thus, benign images may have more prevalent artifacts than malicious images which could be used by the model to learn a suboptimal solution. To prevent this from occurring, we train the same model on a subset of the dataset where the distribution of program size between classes are more similar, known as the "filtered" model. We use a genetic algorithm that minimises the Kullback-Leibler divergence D_{KL} between the distributions of program size for each class of program by modelling it as a Knapsack problem (Khuri et al., 1994). We also encourage the algorithm to select a subset with an approximately equal number of malicious and benign samples to ensure class balance.

We train both the base model, and the filtered model trained for 20 epochs, with the binary cross entropy loss function and a batch size of 32. Training was conducted using an Apple M2 Pro CPU, leveraging the metal performance shaders backend for hardware acceleration, with 16GB of RAM using the AdamW optimiser.

3.6 Obfuscation

In modern malware, obfuscation is highly pervasive. As a result of this, we believe that static malware detection methods learn to identify obfuscation as opposed to other malicious characteristics. However, obfuscation is not exclusive to malicious programs, as legitimate software developers may use similar techniques to protect their intellectual property. Should this be the case, legitimate benign software that have been obfuscated may be misidentified as malicious, thereby degrading the effectiveness of these approaches in real world deployment.

We investigate this by observing how the models performance changes as we increase the presence of obfuscation within the unobfuscated dataset. We perform three trials for each model, that vary which kinds of program are to be obfuscated. For example, only malicious, benign, and all programs. At each step, we obfuscate a certain percentage of programs within the dataset and this percentage increases by 10% upon completion. We collect the logits for each sequence, which are used to calculate both the sequence level, and the file level performance. We also perform this experiment on the image classification model to see if these models are better suited to obfuscation resiliant malware detection.

Obfuscation is performed using a tool known as UPX, which is a binary packing tool. We chose this tool due to its extensive compatability with windows executables. Furthermore, a number of our malicious executables have been identified as using this tool for obfuscation, and it has also been used to compress and obfuscate benign files. Therefore, we feel that it is a suitable tool to replicate the forms of obfuscation used by both malicious and legitimate software developers seen in the wild without having to implement our own obfuscation tooling.

Chapter 4

Results

4.1 Baseline

Algorithm	Feature	Accuracy	F1 Score	Precision	Recall
Support Vector Classifier	DLL	0.89	0.89	0.89	0.89
	Function	0.92	0.92	0.92	0.92
Logistic Regression	DLL	0.87	0.87	0.87	0.87
	Function	0.91	0.91	0.92	0.91
Decision Tree	DLL	0.88	0.88	0.89	0.89
	Function	0.93	0.93	0.93	0.93

Table 4.1: Binary classification metrics of our baseline approaches.

The results of our baseline approaches are presented in table 4.1, and indicate that all methods tested are comparible in terms of performance. When trained on non-unique external function usage, these methods achieve an F1 score of approximately 0.9. External library usage however, incurs a minor degradation in classification performance, as the same methods achieve an F1 score of approximately 0.88. Therefore, we can assume that the usage of external functions is a better indicator of a programs intent.

This can be explained by our earlier analysis of executable similarity based on the same features. We found that the average pairwise similarity between malicious and benign executables based on the function level information was much lower than that of the library level information. Therefore, the API methods used by the programs within our dataset exhibit more variation across benign and malicious samples than external libraries and are thus a better feature for classification.

Our results indicate that the best algorithm for classification was the decision tree classifier, trained on the external function usage of our programs. In addition to providing the best performance of all baseline methods tested in this experiment, decision tree algorithms have the added benefit of being interpretable. This means that we can visualise the internal mechanisms of the algorithm, and reason about how programs are categorised.

Model	Sequence Level				File Level			
	Accuracy	F1-Score	Precision	Recall	Accuracy	F1-Score	Precision	Recall
From Scratch	0.8756	0.8742	0.8798	0.8756	0.8976	0.8971	0.9095	0.8989
From Pretrained	0.9010	0.9002	0.9042	0.9010	0.9452	0.9452	0.9479	0.9458
From Pretrained (w/ Frozen Attention Weights)	0.8302	0.8265	0.8409	0.8302	0.8076	0.8037	0.8393	0.8098

Table 4.2: Performance of our RoBERTa model for opcode sequence classification and malware detection

4.2 Sequence Classification

In table 4.2 we present both the sequence level, and file level classifiction metrics of the models trained in this investigation. We find that in sequence level tasks, where the model is tasked with classifying a subsequence of opcodes as either malicious or benign, the pretrained model offers the best performance. When this pretraining stage is skipped, performance degrades by approximately 3%, indicating the prior understanding of opcode sequences is necessary to more accurately identify malicious sequences. In comparison to our baseline approaches presented in table 4.1, only the pretrained model offers similar performance.

In addition to this, the performance of the same pretrained model is impacted significantly when the parameters of its attention layers are frozen. These weights encode the understanding of langaue developed during the pretraining stage to create an encoded representation of a sequence, which is then used by a classification head to determine the class likelihood values of an opcode sequence. In fine-tuning tasks, these initial layer weights are frozen to reduce the computational cost, and the amount of training data used during training the model. However, in our case freezing of the attention layers has a significantly negative implication on the models performance effect. We believe this is a result of lower model complexity, as only approximately 1% of the models parameters are updated during training. It is well established that the number of a models learnable parameters is directly associated with its performance (Kaplan et al., 2020). By freezing the weights in the attention layers, the number of learnable parameters are reduced, leading to worse performance. (Lee et al., 2019) show that when fine-tuning a model, freezing of the first 75% of layers in similar models incurs only a 10% decrease in performance against its fully-trainable equivalent. Therefore, a better strategy for selecting which layers to freeze in fine-tuning tasks is necessary.

These sequence level tasks however are not representative of our models performance in real world tasks. To understand how they perform in real-world malware identification tasks, we firstly classify each opcode sequence that make up the executable, and take the mean to find the file level class likelihood values. We find that the pretrained model achieves the best performance in file level tasks, with an F1 score of 0.94, outperforming our baseline approaches by a small margin. This increase in performance between sequence level and file level tasks is also seen in the non-pretrained model to a lesser extent. However, the model with frozen attention layers exhibits a decrease in performance on file level tasks as the class logits used to generate the file level logits are more likely to be incorrect, negatively impacting the lieklihood of the model correctly identifying the class of the file.

4.3 Image Classification

Both image classification models demonstrate comparible performance to our baseline approaches, as shown in table 4.3. In addition to this, both models outperform the non-pretrained and the frozen attention weight sequence classification models. However, they are both beaten by the pretrained

Dataset	Accuracy	F1-Score	Precision	Recall
Full	0.9316	0.9315	0.9314	0.9322
Filtered	0.9306	0.9306	0.9309	0.9306

Table 4.3: Performance of the Image Classification Models for Malware detection

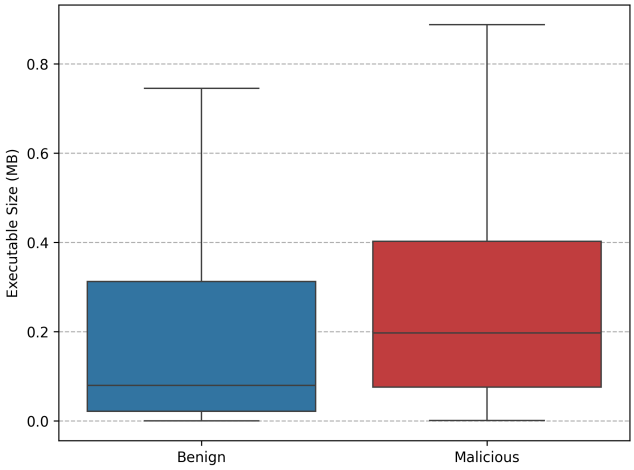


Figure 4.1: Distribution of executable size in the filtered dataset

sequence classification model in file level tasks. Images used by this model contains the entire executable, including the metadata stored in the executable header, static data, and the code itself. Our sequence classification models however, only use the opcodes of the instructions to determine a programs classification. Therefore, pretrained sequence classification models are more accurately identify malware, with less information, and by integrating the aforementioned information used by image classification models, we may be able to further improve its ability to recognise malicious programs.

We have also previously shown that there is a significant discrepancy in the size of executables across classes in our dataset. As we are required to resize images to a constant resolution of 320×320 pixels, artifacts originating from the resizing algorithm may be more pervasive in a single class. This could prevent the model from accurately identifying malware, as it may rely on the presence of these artifacts to distinguish between malicious and benign software. Therefore, we train another model which is trained on a subset of our dataset, which has a more consistent distribution of executable sizes between classes. A genetic algorithm was used to identify this subset, and the resulting collection has distributional distance of zero, and an approximately equal ratio of malicious to benign samples (fig. 4.1). However, the models trained on this dataset demonstrate performance similar to models trained on the full dataset. These results suggest that this is not be the case, as the size of a program has very little impact on its classification.

Furthermore, the architecture of these models make them highly efficient. We are able to achieve comprable performance to that of the sequence classification models, with only approximately 20% of the paramters. This enable these models to be run on devices with lower computational resources, with only a minor decrease in performance. In addition to this, the entire program is classified at once, as opposed to our RoBERTa model, which iterates over each sequence that make up its code. Therefore, this model is much faster to perform inference.

4.4 Obfuscation Experiment

4.4.1 SEQUENCE CLASSIFICATION

We find that our sequence classification models seem to heavily rely on obfuscation as an indicator of a programs intent. This can be seen in Figure 4.2a, where an increased presence of obfuscated malicious samples drives an increase in the models classification performance. When only bengin files are obfuscated however, we see a marginal increase in classification accuracy, suggesting that there may have been a handful of obfuscated samples in our benign dataset. This idea is reinforced by the fact that we see greater classification accuracy when all files are obfusctaed as opposed to when only malicious samples are obfuscated. However, it may also be possible that when we apply obfuscation across all samples in the dataset, there is a higher likelihood of any given sample being obfuscated due to the larger sample size. Therefore, it may be that when we apply obfuscation across all samples, as opposed to a single class, more of a single class has been obfuscated which drives an increase in the classification accuracy, particularly with malicious programs.

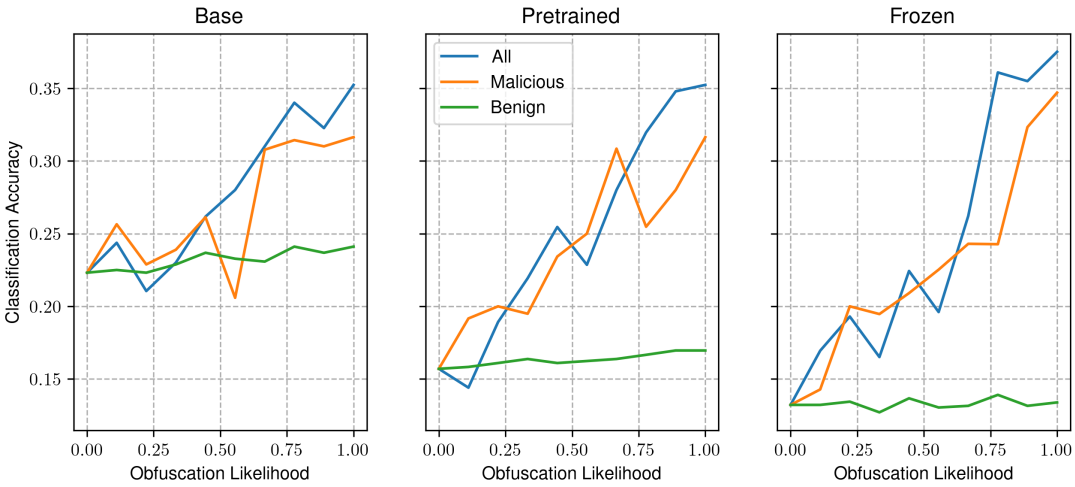
When we consider sequence level accuracy however, this trend is pronounced to a lesser extent, as shown in Figure 4.2b. We do see an a positive correlation between classification accuracy and obfuscation likelihood across all samples, however when obfuscation is applied to a single class of program, we see no meaningful trend.

To better understand how the model deals with obfuscation, we quantify the certainty in the predictions of the model by taking the Kullback-Leibler Divergence eq. (4.1) between the models output, and the uniform distribution. This metric tells us how confident a model is in its predictions, by measuring the distributional distance between its predictions, and a distribution representing the model picking at random. We find that certainty increases significantly across all trials as we increase the prevalance of obfuscation as shown in fig. 4.2c. This suggests that the model considers obfuscation as highly indicative of a samples class.

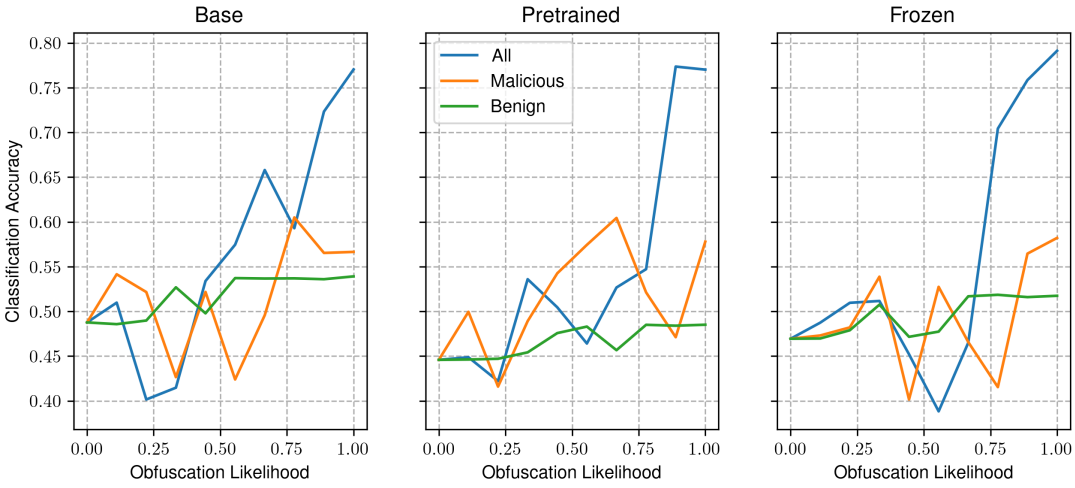
$$D_{KL}(p \parallel q) = \sum_{x \in \mathcal{X}} p(x) \log\left(\frac{p(x)}{q(x)}\right) \tag{4.1}$$

We also see that across all models tested in this experiment, the trends are largely similar. An example of this can be seen in how the file-level classification accuracy increases as we increase the presence of obfuscated malicious samples in the dataset. A similar trend can be seen across all models, however the extent of this varies between them. This suggests that our models are learning similar representations, yet are limited by other factors such as a prior understanding of the langauge, and insufficient model complexity. To visualise this, we perform representational similarity analysis (RSA) on the activations of our models using Centered Kernel Alignment (CKA) as outlined in (Kornblith et al., 2019) (shown in appendix A.5). This metric quantifies the correspondance between hidden representations of data between models trained from different representations, which is visualised in fig. 4.3. This shows how the similarity of representations in the attention layers of the pretrained and non-pretrained models decrease with the number of layers, and the demonstrate how the representation at the classification head are very similar. The model with frozen attention layers however, exhibits a greater similarity across the representations at all layers. However, the first layers seem to be most disimilar between the two. This suggests that the model pretrained model has most updates to its representations at the initial layers. Furthermore, the similarity in representations at the classification head of the frozen model are highly similar to the internal representations, suggesting that the frozen model is limited by its insufficient complexity, despite learning similar representations of information.

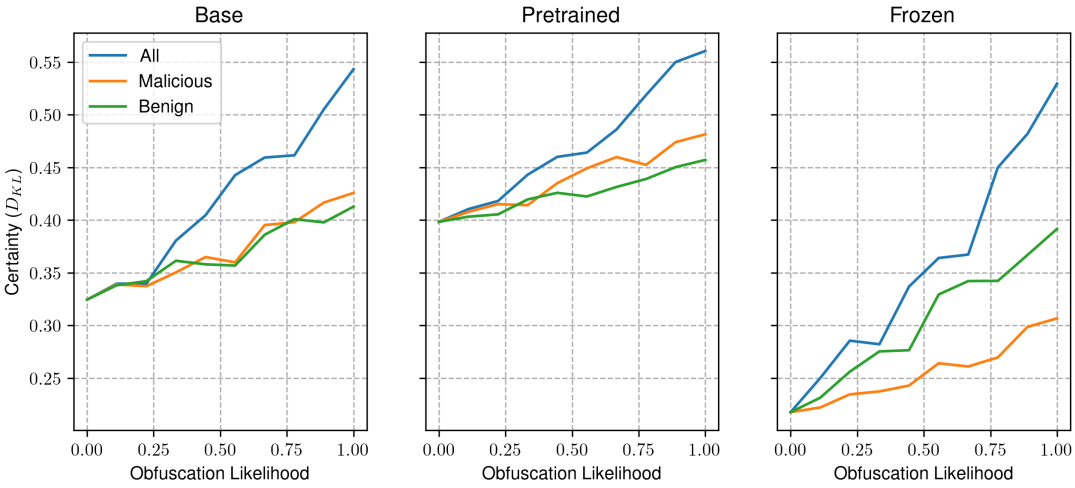
We then repeat this experiment on two groups of models trained on modified datasets to see if data augmentation can produce models which are more resiliant to obfuscation. The first of these datasets has had all of its benign samples obfuscated prior to opcode extraction, and the second



(a) File-level classification metrics

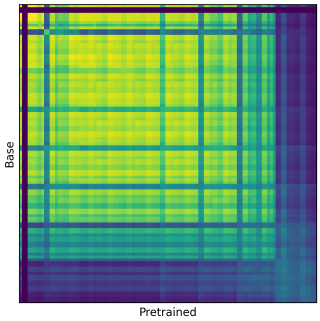


(b) Sequence level classification metrics

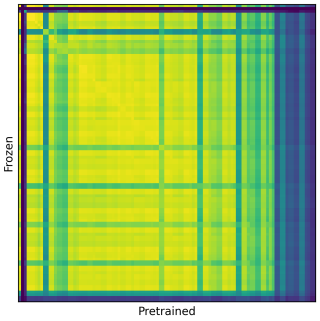


(c) Confidence in model predictions

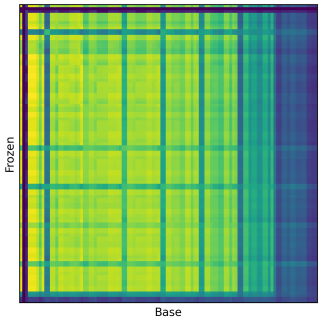
Figure 4.2: Impact of obfuscation on the performance of file-level and sequence-level classification, and certainty of our RoBERTa sequence classification models.



(a) RSA of the pretrained and non-pretrained models.



(b) RSA of the pretrained model and our pretrained model with frozen attention layer weights.



(c) RSA of the pretrained model with frozen attention layer weights and the non-pretrained model.

Figure 4.3: Representational Similarity Analysis (RSA) of our baseline models using Centered Kernel Alignment.

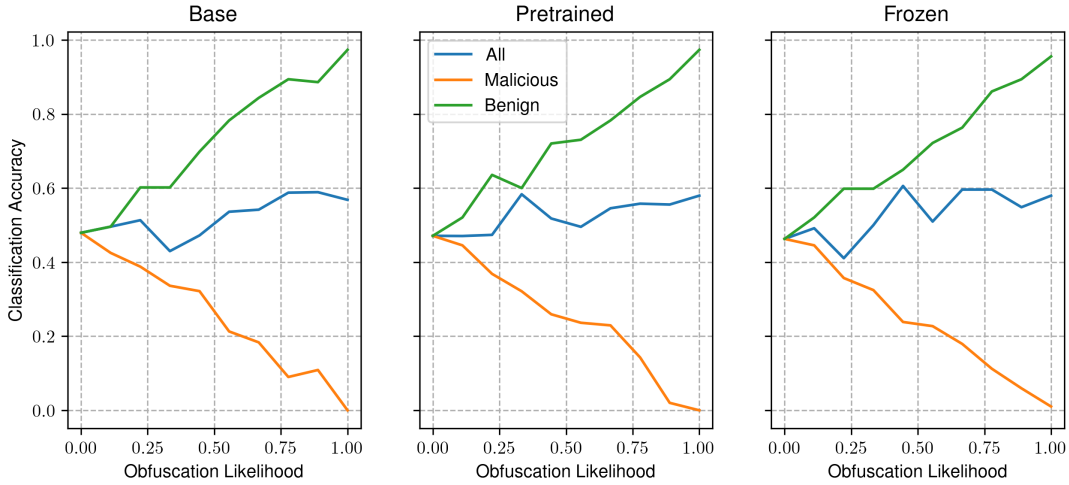
contains both the obfuscated and regular benign samples. We refer to these as our obfuscated bengin and partially obfuscated datasets respectively.

Obfuscated Benign

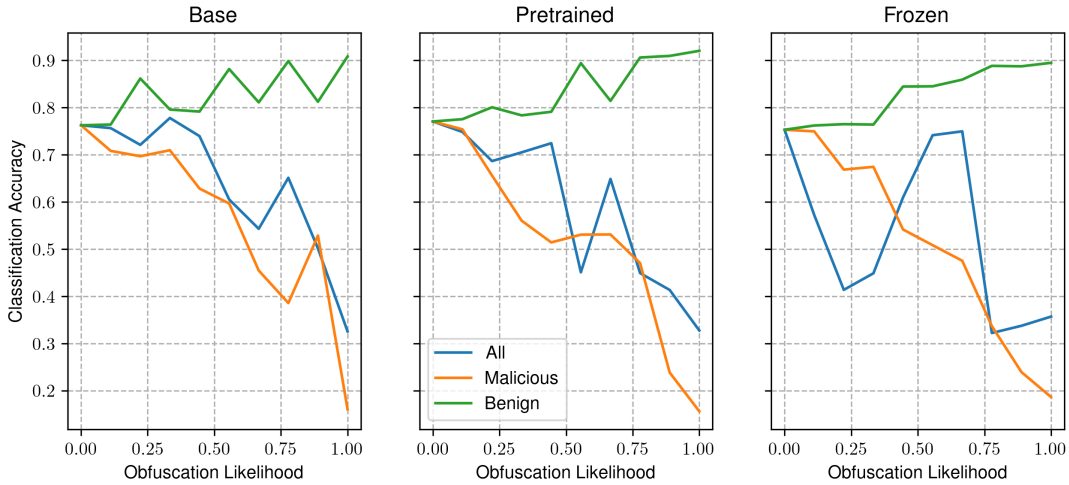
The models trained on the this dataset all demonstrate increased classification accuracy when only bengin samples are obfuscated. When only malicious programs are obfuscated however, its accuracy decreases. In addition to this, when all samples are targets for obfuscation, the accuracy remains relatively consistent as shown in fig. 4.4a. This is to be expected, as the obfuscated benign samples used to train the model are generated using UPX, the same tool used to generate obfuscated samples in our experiment. Therefore, the model learns to recognise this form of obfuscation, as it is indicative of benign samples according to its training data. When all samples are obfuscated, the model is not able to accurately distinguish between the classes in the dataset, as it expects only benign samples to be obfuscated. Thus, when both classes are obfuscated, it is effectively guessing which class it belongs to. This can also be seen in its certainty, where the KL-Divergence decreases significantly as we increase the presence of obfuscation as shown in fig. 4.4c, suggesting the predicted class distribution approaches random selection with increased presence of obfuscation across both classes.

Partially Obfuscated Benign

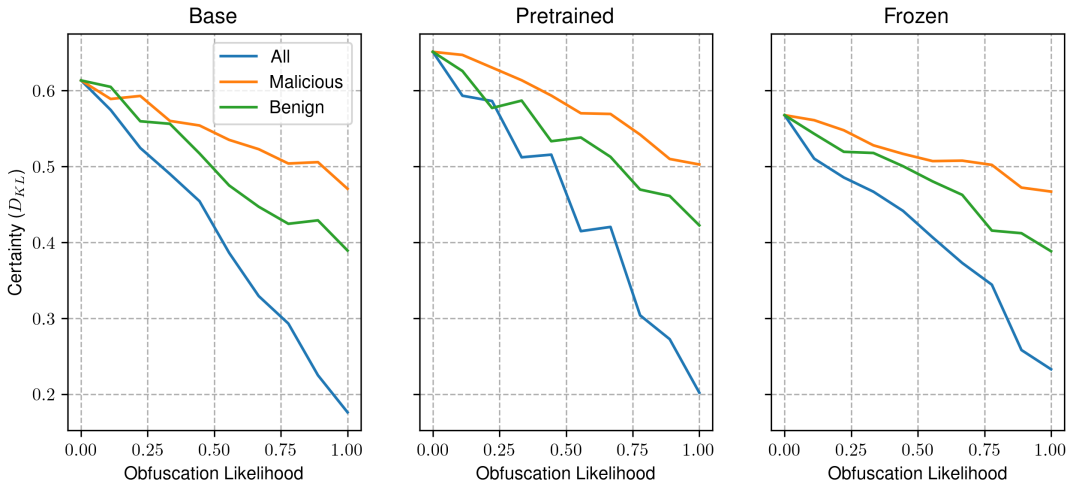
In comparison to the models trained on our baseline, and obfuscated benign datasets, the results of the models trained on the partially obfuscated benign dataset demonstrate significant variation



(a) File-level classification metrics

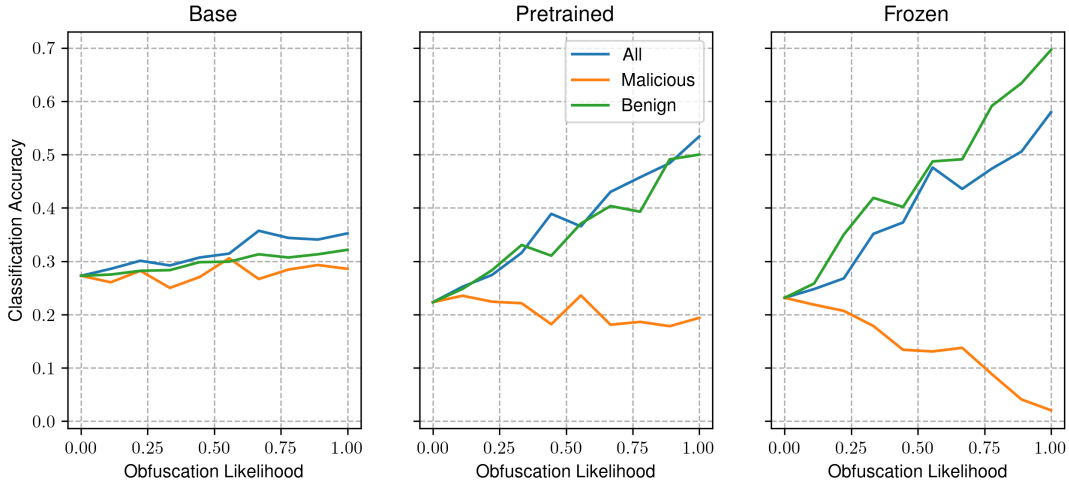


(b) Sequence level classification metrics

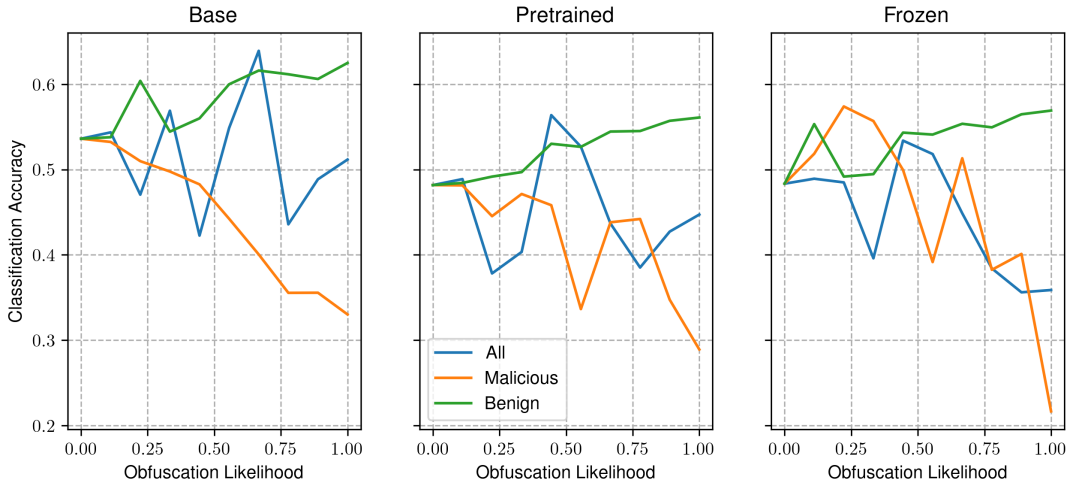


(c) Confidence in model predictions

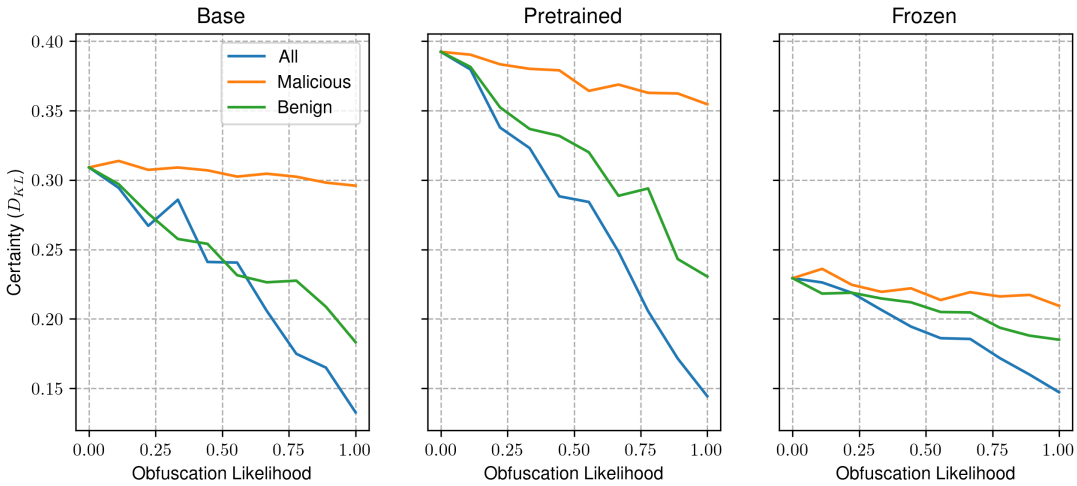
Figure 4.4: Impact of obfuscation on the performance of file-level and sequence-level classification, and certainty of our RoBERTa sequence classification models trained on the obfuscated bengin dataset.



(a) File-level classification metrics



(b) Sequence level classification metrics



(c) Certainty (KL-Divergence between logits and uniform distribution)

Figure 4.5: Impact of obfuscation on the performance of file-level and sequence-level classification, and certainty of our RoBERTa sequence classification models trained on the partially obfuscated benign dataset.

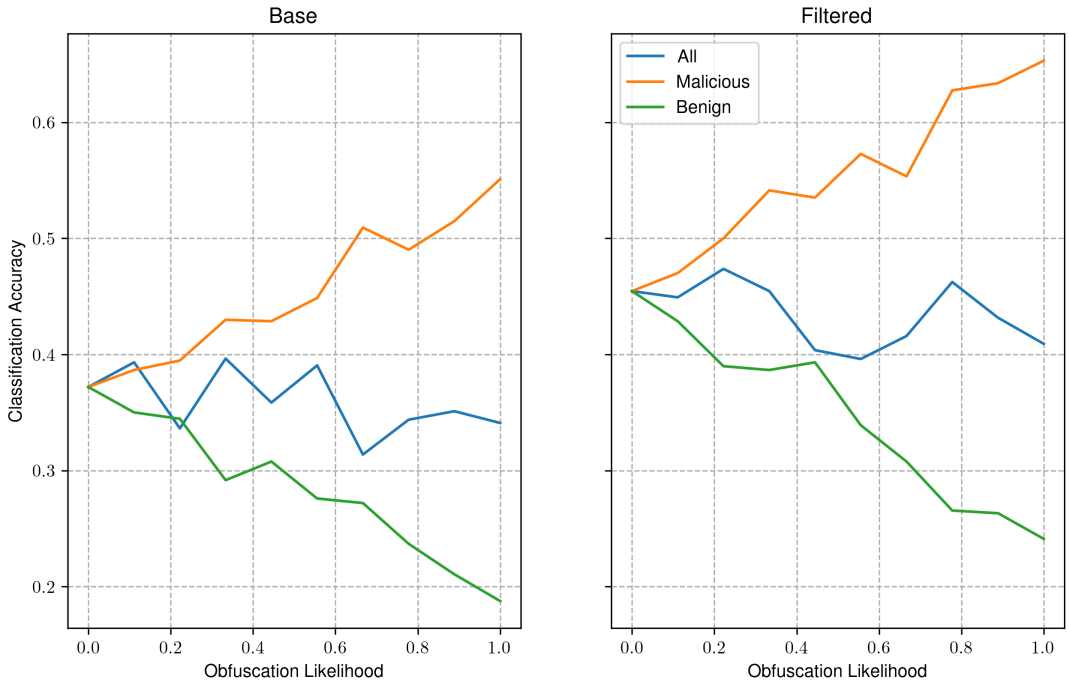
between eachother. Our previous models show that models trained on the same dataset, exhibit similar trends in performance as we increase the presence of obfuscation. However, the model trained to identify malware with no pretraining step shows that classification accuracy increases slightly with the prevalance of obfuscation across all obfuscation targets. The pretrained, and frozen attention weight models however, increase significantly as obfuscated benign samples increase, while obfuscated malicious samples have an adverse impact on its accuracy. This suggests that the non-pretrained model learns to identify malware beyond the presence of obfuscation. The other models that include a pretraining stage however, have both identified obfuscation as an indicator of benign programs.

In addition to this, we also see that these models also become increasingly uncertain as we increase the presence of obfuscation. Our obfuscated benign models also demonstrate this behavior, indicating that obfuscated benign samples in the training dataset leads to less accurate predictions.

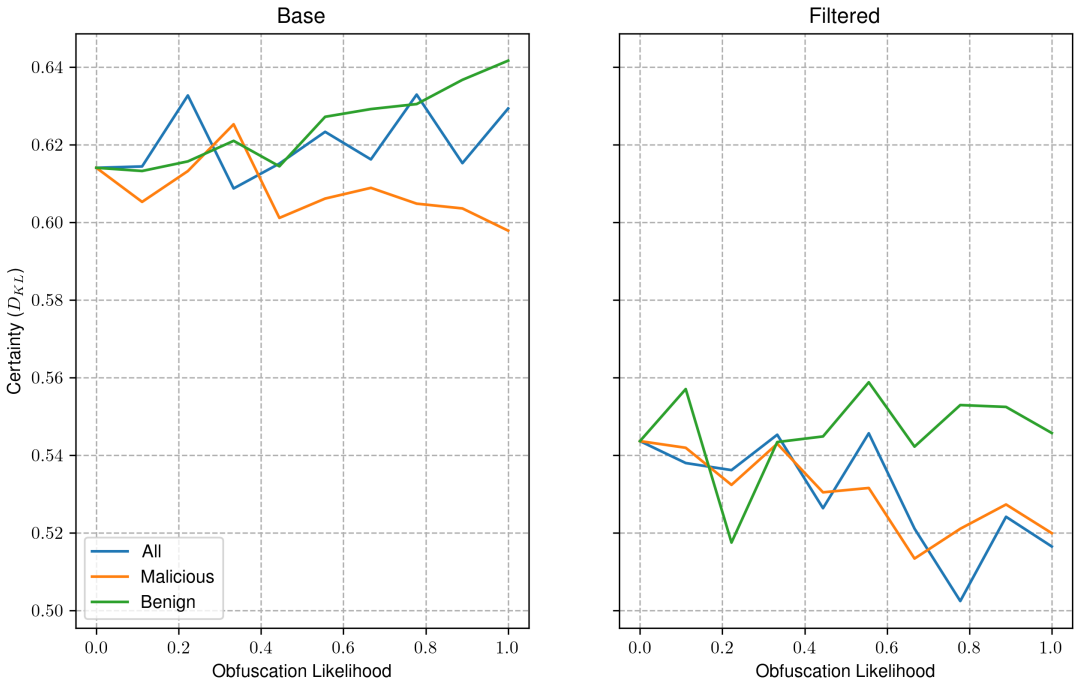
4.4.2 IMAGE CLASSIFER

The impact of obfuscation on our image classification models is shown in Figure 4.6. We find that both of the image based malware detection models are more likely to identify malicious software based on the presence of obfuscation. In both cases, we see that the presence of obfuscation increases across our malicious samples is positively correlated with the classification accuracy. In comparison to this, an increased prevalance of obfuscation in benign samples leads to a degradation in performance. Furthermore, when obfuscation is applied across all samples in this dataset, we see no meaningful trend in classification accuracy as obfuscation becomes more common.

This heavily suggests that these models use the presence of obfuscation as an indicator of an executables kind. The certainty of the base model's predictions seem to indicate



(a) Classification Accuracy



(b) Confidence in model predictions

Figure 4.6: Impact of obfuscation on the performance of image-based malware detection models, and the confidence of the model in its predictions.

Chapter 5

Discussion

Word Count: 3000

- Are dynamic methods vulnerable to advanced obfuscation techniques?
  - It's possible to hide API calls from standard API call hooking techniques
  - Other redundant API calls may be made to obfuscate call sequences
  - Advanced malware (i.e. Rootkits) are able to stealthily perform malicious actions on the host
- Are opcode sequences suitable features for NLP based malware detection?
  - Full disassembly may offer provide better information for transformer architectures
  - Was not tested due to time/compute limitations
  - Byte-level modelling may work, but the presence of polymorphic malware and encrypted payloads may be challenging to model accurately
- Is executable packing (UPX) a good proxy for the obfuscation methods seen in the wild?
  - What other obfuscation techniques and solutions could work for this?
- Could diffusion modelling enable machine de-obfuscation?
  - Obfuscation is analgous to noise in a signal
  - Diffusion models are well suited to denoising of data
  - Might be able to de-obfuscate programs to generate large scale
- Data limitations
  - Lack of large-scale, benign executable datasets
  - High presence of obfuscation in real-world malware samples
  - Unobfuscated malware samples are not representative of malware seen in the wild as they are often "toy" projects, and lack advanced capabilities
  - Unobfuscated, advanced malware are hard to find
- Issues with Obfuscation Experiment
  - single trial and random selection of obfuscated samples may produce less meaningful results
  - trials should be repeated to see if this changes things
  - trials took a long time to run so only a single run was possible.
- Conclusion

`./discussion.tex` does not exist

Bibliography

A. Saeed, Imtithal, Ali Selamat and Ali M. A. Abuagoub (Apr. 2013). 'A Survey on Malware and Malware Detection Systems'. In: *International Journal of Computer Applications* 67.16, pp. 25–31. ISSN: 09758887. DOI: 10.5120/11480-7108. (Visited on 29/07/2025).

Aboaoja, Faitouri A. et al. (Jan. 2022). 'Malware Detection Issues, Challenges, and Future Directions: A Survey'. In: *Applied Sciences* 12.17, p. 8482. ISSN: 2076-3417. DOI: 10.3390/app12178482. (Visited on 10/12/2024).

Ainapure, Kaushik and Microsoft (Jan. 2025). *Dynamic Link Library (DLL)*. https://learn.microsoft.com/en-us/troubleshoot/windows-client/setup-upgrade-and-drivers/dynamic-link-library. (Visited on 15/08/2025).

Akbanov, Maxat, Vassilios G Vassilakis and Michael D Logothetis (2019). 'WannaCry Ransomware: Analysis of Infection, Persistence, Recovery Prevention and Propagation Mechanisms'. In: *Journal of Telecommunications and Information Technology* 1, pp. 113–124.

Alenezi, Mohammed et al. (Dec. 2020). 'Evolution of Malware Threats and Techniques: A Review'. In: *International Journal of Communication Networks and Information Security* 12, p. 326. DOI: 10.17762/ijcnis.v12i3.4723.

Anderson, Hyrum S. and Phil Roth (Apr. 2018). *EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models*. DOI: 10.48550/arXiv.1804.04637. arXiv: 1804.04637 [cs]. (Visited on 05/08/2025).

Aslan, Ömer and Abdullah Asim Yilmaz (2021). 'A New Malware Classification Framework Based on Deep Learning Algorithms'. In: *IEEE Access* 9, pp. 87936–87951. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3089586. (Visited on 28/07/2025).

Attaluri, Srilatha, Scott McGhee and Mark Stamp (May 2009). 'Profile Hidden Markov Models and Metamorphic Virus Detection'. In: *Journal in Computer Virology* 5.2, pp. 151–169. ISSN: 1772-9904. DOI: 10.1007/s11416-008-0105-1. (Visited on 25/02/2025).

AV-TEST - The Independent IT-Security Institute (2025). *AV-ATLAS - Malware & PUA*. https://portal.av-atlas.org/malware. (Visited on 19/12/2024).

Bacci, Alessandro et al. (2018). 'Impact of Code Obfuscation on Android Malware Detection Based onStatic and Dynamic Analysis'. In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy*. Funchal, Madeira, Portugal: SCITEPRESS - Science and Technology Publications. DOI: 10.5220/0006642503790385. (Visited on 21/07/2025).

Baezner, Marie and Patrice Robin (Oct. 2017). *Stuxnet*. Report 4. ETH Zurich. DOI: 10.3929/ethz-b-000200661. (Visited on 22/07/2025).

Balakrishnan, Arini and Chloe Schulze (Dec. 2005). 'Code Obfuscation Literature Survey'. In.

Bridge, Karl and Microsoft (July 2025). *PE Format*. https://learn.microsoft.com/en-us/windows/win32/debug/pe-format. (Visited on 15/08/2025).

British Computing Society (June 2022). *BCS Code of Conduct*. https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/. (Visited on 14/08/2025).

Christodorescu, Mihai and Somesh Jha (Mar. 2004). 'Static Analysis of Executables to Detect Malicious Patterns'. In: 12.

Cochran, Kodi A. (2024). 'The CIA Triad: Safeguarding Data in the Digital Realm'. In: *Cybersecurity Essentials: Practical Tools for Today's Digital Defenders*. Ed. by Kodi A. Cochran. Berkeley, CA:

Apress, pp. 17–32. ISBN: 979-8-8688-0432-8. DOI: 10.1007/979-8-8688-0432-8_2. (Visited on 22/07/2025).

Damodaran, Anusha et al. (Feb. 2017). 'A Comparison of Static, Dynamic, and Hybrid Analysis for Malware Detection'. In: *Journal of Computer Virology and Hacking Techniques* 13.1, pp. 1–12. ISSN: 2263-8733. DOI: 10.1007/s11416-015-0261-z. (Visited on 29/07/2025).

Darem, Abdulbasit et al. (Dec. 2021). 'Visualization and Deep-Learning-Based Malware Variant Detection Using OpCode-level Features'. In: *Future Generation Computer Systems* 125, pp. 314–323. ISSN: 0167-739X. DOI: 10.1016/j.future.2021.06.032. (Visited on 28/07/2025).

Demırcı, Denız et al. (2022). 'Static Malware Detection Using Stacked BiLSTM and GPT-2'. In: *IEEE Access* 10, pp. 58488–58502. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2022.3179384. (Visited on 25/02/2025).

Devlin, Jacob et al. (May 2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. DOI: 10.48550/arXiv.1810.04805. arXiv: 1810.04805 [cs]. (Visited on 28/02/2025).

European Union Agency for Cybersecurity (2024). *ENISA Threat Landscape 2024: July 2023 to June 2024*. LU: Publications Office. (Visited on 21/02/2025).

Habibi, Omar, Mohammed Chemmakha and Mohamed Lazaar (Aug. 2023). 'Performance Evaluation of CNN and Pre-trained Models for Malware Classification'. In: *Arabian Journal for Science and Engineering* 48.8, pp. 10355–10369. ISSN: 2191-4281. DOI: 10.1007/s13369-023-07608-z. (Visited on 25/02/2025).

Idika, Mathur (Feb. 2007). 'A Survey of Malware Detection Techniques'. In.

Jain, M., W. Andreopoulos and M. Stamp (2020). 'Convolutional Neural Networks and Extreme Learning Machines for Malware Classification'. In: *Journal of Computer Virology and Hacking Techniques* 16.3, pp. 229–244. DOI: 10.1007/s11416-020-00354-y.

Kakisim, Arzu Gorgulu, Sibel Gulmez and Ibrahim Sogukpinar (Mar. 2022). 'Sequential Opcode Embedding-Based Malware Detection Method'. In: *Computers & Electrical Engineering* 98, p. 107703. ISSN: 0045-7906. DOI: 10.1016/j.compeleceng.2022.107703. (Visited on 28/07/2025).

Kamboj, Akshit et al. (Mar. 2023). 'Detection of Malware in Downloaded Files Using Various Machine Learning Models'. In: *Egyptian Informatics Journal* 24.1, pp. 81–94. ISSN: 1110-8665. DOI: 10.1016/j.eij.2022.12.002. (Visited on 28/07/2025).

Kaplan, Jared et al. (Jan. 2020). *Scaling Laws for Neural Language Models*. DOI: 10.48550/arXiv.2001.08361. arXiv: 2001.08361 [cs]. (Visited on 13/02/2025).

Karbab, ElMouatez Billah et al. (Mar. 2018). 'MalDozer: Automatic Framework for Android Malware Detection Using Deep Learning'. In: *Digital Investigation* 24, S48–S59. ISSN: 1742-2876. DOI: 10.1016/j.diin.2018.01.007. (Visited on 20/11/2024).

Khuri, Sami, Thomas Bäck and Jörg Heitkötter (Apr. 1994). 'The Zero/One Multiple Knapsack Problem and Genetic Algorithms'. In: *Proceedings of the 1994 ACM Symposium on Applied Computing*. SAC '94. New York, NY, USA: Association for Computing Machinery, pp. 188–193. ISBN: 978-0-89791-647-9. DOI: 10.1145/326619.326694. (Visited on 04/08/2025).

Ki, Youngjoon, Eunjin Kim and Huy Kang Kim (June 2015). 'A Novel Approach to Detect Malware Based on API Call Sequence Analysis'. In: *International Journal of Distributed Sensor Networks* 11.6, p. 659101. ISSN: 1550-1329. DOI: 10.1155/2015/659101. (Visited on 02/11/2024).

Kornblith, Simon et al. (July 2019). *Similarity of Neural Network Representations Revisited*. DOI: 10.48550/arXiv.1905.00414. arXiv: 1905.00414 [cs]. (Visited on 12/08/2025).

Kramer, Simon and Julian C. Bradfield (May 2010). 'A General Definition of Malware'. In: *Journal in Computer Virology* 6.2, pp. 105–114. ISSN: 1772-9904. DOI: 10.1007/s11416-009-0137-1. (Visited on 21/02/2025).

Lee, Jaejun, Raphael Tang and Jimmy Lin (Nov. 2019). *What Would Elsa Do? Freezing Layers During Transformer Fine-Tuning*. DOI: 10.48550/arXiv.1911.03090. arXiv: 1911.03090 [cs]. (Visited on 12/08/2025).

Li, Ce et al. (Nov. 2022). 'DMalNet: Dynamic Malware Analysis Based on API Feature Engineering and Graph Learning'. In: *Computers & Security* 122, p. 102872. ISSN: 0167-4048. DOI: 10.1016/j.cose.2022.102872. (Visited on 27/11/2024).

Li, Yang et al. (Jan. 2023). 'APIASO: A Novel API Call Obfuscation Technique Based on Address Space Obscurity'. In: *Applied Sciences* 13.16, p. 9056. ISSN: 2076-3417. DOI: 10.3390/app13169056. (Visited on 25/02/2025).

M., Gopinath and Sibi Chakkaravarthy Sethuraman (Feb. 2023). 'A Comprehensive Survey on Deep Learning Based Malware Detection Techniques'. In: *Computer Science Review* 47, p. 100529. ISSN: 1574-0137. DOI: 10.1016/j.cosrev.2022.100529. (Visited on 20/11/2024).

Maniriho, Pascal, Abdun Naser Mahmood and Mohammad Jabed Morshed Chowdhury (Mar. 2024). 'A Systematic Literature Review on Windows Malware Detection: Techniques, Research Issues, and Future Directions'. In: *Journal of Systems and Software* 209, p. 111921. ISSN: 0164-1212. DOI: 10.1016/j.jss.2023.111921. (Visited on 18/02/2025).

Merabet, Hoda El and Abderrahmane Hajraoui (2019). 'A Survey of Malware Detection Techniques Based on Machine Learning'. In: *International Journal of Advanced Computer Science and Applications* 10.1. ISSN: 21565570, 2158107X. DOI: 10.14569/IJACSA.2019.0100148. (Visited on 20/11/2024).

National Audit Office (NAO) (Oct. 2017). *Investigation: WannaCry Cyber Attack and the NHS*. Tech. rep. London, England: National Audit Office.

Ni, Sang, Quan Qian and Rui Zhang (Aug. 2018). 'Malware Identification Using Visualization Images and Deep Learning'. In: *Computers & Security* 77, pp. 871–885. ISSN: 0167-4048. DOI: 10.1016/j.cose.2018.04.005. (Visited on 21/02/2025).

Quiquet, François (Oct. 2023). *An analysis of the Viasat cyber attack with the MITRE ATT&CK® framework*. (Visited on 21/07/2025).

Radare Organization (July 2025). *Radare2*. radare org. (Visited on 30/07/2025).

Radford, Alec et al. (n.d.). 'Language Models Are Unsupervised Multitask Learners'. In: ().

Rahali, Abir and Moulay A. Akhloufi (Oct. 2021). 'MalBERT: Malware Detection Using Bidirectional Encoder Representations from Transformers'. In: *2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pp. 3226–3231. DOI: 10.1109/SMC52423.2021.9659287. (Visited on 25/02/2025).

Rizin Organization (July 2025). *Rizin*. Rizin Organization. (Visited on 30/07/2025).

Rokon, Md Omar Faruk et al. (May 2020). *SourceFinder: Finding Malware Source-Code from Publicly Available Repositories*. DOI: 10.48550/arXiv.2005.14311. arXiv: 2005.14311 [cs]. (Visited on 31/07/2025).

Runwal, Neha, Richard M. Low and Mark Stamp (May 2012). 'Opcode Graph Similarity and Metamorphic Detection'. In: *Journal in Computer Virology* 8.1, pp. 37–52. ISSN: 1772-9904. DOI: 10.1007/s11416-012-0160-5. (Visited on 25/02/2025).

Schultz, M.G. et al. (May 2001). 'Data Mining Methods for Detection of New Malicious Executables'. In: *Proceedings 2001 IEEE Symposium on Security and Privacy. S&P 2001*, pp. 38–49. DOI: 10.1109/SECPRI.2001.924286. (Visited on 21/02/2025).

Sharma, Arindam, Pasquale Malacaria and MHR Khouzani (June 2019). 'Malware Detection Using 1-Dimensional Convolutional Neural Networks'. In: *2019 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 247–256. DOI: 10.1109/EuroSPW.2019.00034. (Visited on 28/07/2025).

Singh, Jagsir and Jaswinder Singh (May 2020). 'Detection of Malicious Software by Analyzing the Behavioral Artifacts Using Machine Learning Algorithms'. In: *Information and Software Technology* 121, p. 106273. ISSN: 0950-5849. DOI: 10.1016/j.infsof.2020.106273. (Visited on 28/07/2025).

Tan, Mingxing and Quoc V. Le (Sept. 2020). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. DOI: 10.48550/arXiv.1905.11946. arXiv: 1905.11946 [cs]. (Visited on 31/07/2025).

Tian, Ronghua et al. (2009). 'An Automated Classification System Based on the Strings of Trojan and Virus Families'. In: *2009 4th International Conference on Malicious and Unwanted Software (MALWARE)*, pp. 23–30. DOI: 10.1109/MALWARE.2009.5403021.

Vaswani, Ashish et al. (Aug. 2023). *Attention Is All You Need*. DOI: 10.48550/arXiv.1706.03762. arXiv: 1706.03762 [cs]. (Visited on 30/07/2025).

Wang, Haojun et al. (2021). 'Deep Learning and Regularization Algorithms for Malicious Code Classification'. In: *IEEE Access* 9, pp. 91512–91523. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3090464. (Visited on 28/07/2025).

Yang, Limin et al. (May 2021). 'BODMAS: An Open Dataset for Learning Based Temporal Analysis of PE Malware'. In: *2021 IEEE Security and Privacy Workshops (SPW)*. San Francisco, CA, USA: IEEE, pp. 78–84. ISBN: 978-1-6654-3732-5. DOI: 10.1109/SPW53761.2021.00020. (Visited on 05/08/2025).

You, Ilsun and Kangbin Yim (2010). 'Malware Obfuscation Techniques: A Brief Survey'. In: *2010 International Conference on Broadband, Wireless Computing, Communication and Applications*, pp. 297–300. DOI: 10.1109/BWCCA.2010.85.

Appendix A

Supplimental Information

A.1 Code Availability

All of the code used in this investigation are publicly available on our GitHub repository[1]. The dataset can be made available upon request, however due to copyright limitations we are unable to distribute benign samples.

A.2 Disassembly Extraction and Processing

Algorithm 1 Extract Disassembly from an Executable

Input: Path to executable P
Output: Sequence of disassembled instructions of P

$L \leftarrow$ empty list
Load P into radare2
Perform full analysis ▷ R2 Command: `aaa`
if Instruction set of P is not x86 **then**
    return
end if
$S \leftarrow$ list of sections in P ▷ R2 Command: `iS`
$S_x \leftarrow$ filter S for executable sections
for all $s \in S_x$ **do**
    $o \leftarrow$ section offset of s
    $n \leftarrow$ section size of s
    $D \leftarrow$ disassemble bytes from o to $o + n$ ▷ R2 Command: `pda` o `@` $o + n$
    $D' \leftarrow$ filtered D to remove padding bytes
    Append D' to L
end for
return L

[1] `https://github.com/Henry-Ash-Williams/masters-thesis`

Algorithm 2 Extract Opcodes from Assembly Instructions

Input: List of assembly instructions I
Output: Sequence of opcodes

1: $L \leftarrow$ empty list
2: **for all** $i \in I$ **do**
3: $\quad i' \leftarrow$ split i on spaces
4: $\quad$ Append i'_0 to L
5: **end for**
6: **return** L

A.3 Conversion of Executable Programs to Greyscale Images

Algorithm 3 Converts a program to an image

Input: Program $P \in [0, 255]^L$ of L bytes
Output: Matrix $M \in [0, 255]^{d \times d}$

1: $d \leftarrow \lceil \sqrt{L} \rceil$
2: Initialize $V \in [0, 255]^{d^2}$ with zeros
3: **for** $i \leftarrow 1$ to L **do**
4: $\quad V_i \leftarrow P_i$
5: **end for**
6: Reshape V into matrix $M \in [0, 255]^{d \times d}$ in row-major order
7: **return** M

A.4 Genetic Algorithm

Algorithm 4 Apply a random mutation to a genotype

Input: Genome $G \in \{0, 1\}^N$ of N binary values, mutation rate $0 < p \leq 1$
Output: Mutated Genome $G' \in \{0, 1\}^N$

1: $V \in \mathbb{R}^N \leftarrow$ a vector of random numbers in $[0, 1]$
2: $M \in \{0, 1\}^N \leftarrow$ a binary vector where $V_i < p$
3: $G' \leftarrow G \oplus M$
4: **return** G'

Algorithm 5 Compute the fitness of a genotype

Input: Genome $G \in \{0,1\}^N$ of N binary values
Output: The fitness $f \in \mathbb{R}$ of G

$N_m \leftarrow$ Number of malicious samples in G
$N_b \leftarrow$ Number of benign samples in G
$\mu_m, \sigma_m \leftarrow$ mean and variance of the size of malicious programs in G
$\mu_b, \sigma_b \leftarrow$ mean and variance of the size of benign programs in G
$d \leftarrow$ the Kullback-Leibler Divergence between $p_m \sim \mathcal{N}(\mu_m, \sigma_m)$ and $p_b \sim \mathcal{N}(\mu_b, \sigma_b)$
$r \leftarrow \text{clip}\left(1 - \frac{N_b}{N_m}, 0, 1\right)$
$w_d \leftarrow -\left(\frac{e^{-d}}{1+e^{-d}}\right)$
$w_r \leftarrow 1 - w_d$
$f \leftarrow w_d d + w_r r$
return f

Algorithm 6 Simulate Evolution in a Population

Input: The fittest member of the previous population $G \in \{0,1\}^N$, Population size N_p, Mutation rate p_m
Output: The fittest member of the current population $G' \in \{0,1\}^N$

$P \leftarrow N_p$ Copies of G
$F_P \leftarrow$ Fitness of current population ▷ Algorithm 5
$O \leftarrow$ Mutated population of offspring from P with some mutation rate p_m ▷ Algorithm 4
$F_O \leftarrow$ Fitness of offspring population ▷ Algorithm 5
$P' \leftarrow$ New population where the fittest of a parent and its offspring survives
$G' \leftarrow$ The fittest member of P'
return G'

Algorithm 7 Simulate the process of evolution over multiple generations

Input: Number of generations to simulate N_g
Output: The fittest member of the last generation G

$G_0 \in \{0,1\}^N \leftarrow$ initial random genotype
for $i \leftarrow 0$ to N_g **do**
    $G_{i+1} \leftarrow$ Simulate a population with G_i ▷ Algorithm 6
    $F_i \leftarrow$ Fitness of G_{i+1} ▷ Algorithm 5
    if F_i has not decreased in the past 10 generations **then**
        return G_{i+1}
    end if
end for
return G_i

A.5 Centered Kernel Alignment

Given two matrices of n examples of the output from layers in two neural networks trained from different initialisations, $\mathbf{X} \in \mathbb{R}^{n \times d_x}$, and $\mathbf{Y} \in \mathbb{R}^{n \times d_y}$, the Centered Kernel Alignment between these representations is defined as follows:

$$\text{CKA}(\mathbf{K}, \mathbf{L}) = \frac{\text{HSIC}(\mathbf{K}, \mathbf{L})}{\sqrt{\text{HSIC}(\mathbf{K}, \mathbf{K}) \cdot \text{HSIC}(\mathbf{L}, \mathbf{L})}} \tag{A.1}$$

Where HSIC refers to the Hilbert-Schmidt independence criterion, defined in eq. (A.2), $\mathbf{K} = \mathbf{X}\mathbf{X}^T$, and $\mathbf{L} = \mathbf{Y}\mathbf{Y}^T$.

$$\text{HSIC}(\mathbf{K}, \mathbf{L}) = \frac{\text{tr}(\mathbf{KHLH})}{(n-1)^2} \tag{A.2}$$

Where $\mathbf{H} = \mathbf{I}_n - \frac{1}{n}\mathbf{J}_n$, and $\mathbf{J}_n$ is an $n \times n$ matrix of ones.

A.6 Windows Portable Executable File Format

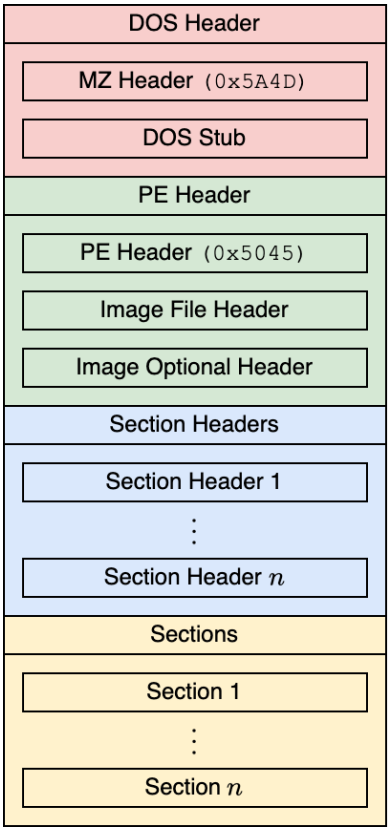


Figure A.1: A high level overview of the windows portable executable file format.

Windows executables are defined in the portable executable file format. These files are not written by developers themselves as they require expert knowledge of low level programming, and instead are generated by a compiler. A high level overview of this file format is shown in Figure A.1, and it has been in use since Windows 95, making it incredibly widespread throughout the internet.

The file begins with a DOS (Disk Operating System) header for compatibility reasons (Bridge et al., 2025), which is followed by the PE header, containing the information necessary for the programs execution. This includes the number of sections and symbols, and the instruction set used by the program. The section headers contain further metadata about the sections which comprise the program, and reference their location in memory. It also stores a pointer to a table referencing any external functions used by the program, such as the Windows API, and other dynamically linked objects, known as DLLs (Ainapure et al., 2025).

Appendix B

Supplimental Results

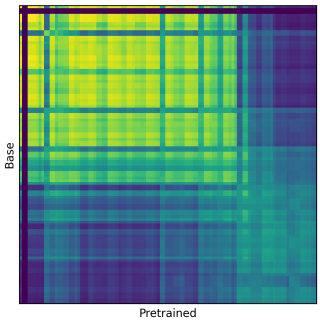
B.1 Obfuscated Benign Sequence Classification Metrics

Model	Sequence Level				File Level			
	Accuracy	F1-Score	Precision	Recall	Accuracy	F1-Score	Precision	Recall
From Scratch	0.8584	0.8587	0.8596	0.8584				
From Pretrained	0.8704	0.8707	0.8721	0.8704				
From Pretrained (w/ Frozen Attention Weights)	0.8471	0.8471	0.8472	0.8471				

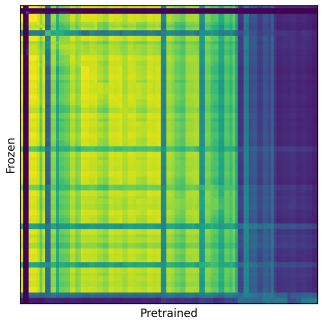
B.2 Partially Obfuscated Benign Sequence Classification Metrics

Model	Sequence Level				File Level			
	Accuracy	F1-Score	Precision	Recall	Accuracy	F1-Score	Precision	Recall
From Scratch	0.8282	0.8256	0.8310	0.8282				
From Pretrained	0.8494	0.8481	0.8503	0.8494				
From Pretrained (w/ Frozen Attention Weights)	0.8053	0.7987	0.8169	0.8053				

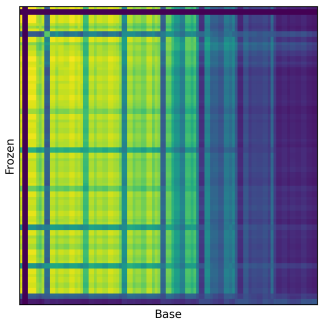
B.3 Representational Similarity Analysis of Obfuscated Benign Sequence Classification Models



(a) RSA of the pretrained and non-pretrained models.



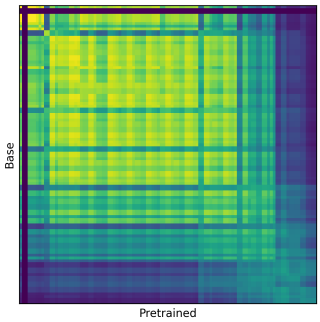
(b) RSA of the pretrained model and our pretrained model with frozen attention layer weights.



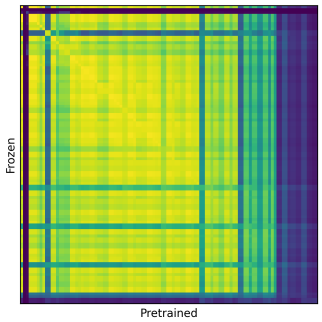
(c) RSA of the pretrained model with frozen attention layer weights and the non-pretrained model.

Figure B.1: Representational Similarity Analysis (RSA) of our obfuscated benign models using Centered Kernel Alignment.

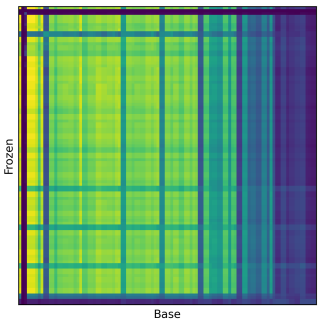
B.4 Representational Similarity Analysis of Partially Obfuscated Benign Sequence Classification Models



(a) RSA of the pretrained and non-pretrained models.



(b) RSA of the pretrained model and our pretrained model with frozen attention layer weights.



(c) RSA of the pretrained model with frozen attention layer weights and the non-pretrained model.

Figure B.2: Representational Similarity Analysis (RSA) of our partially obfuscated benign models using Centered Kernel Alignment.